BloodDrop

A local blood & emergency donor alert network — project report

Live application	blooddrop-sigma.vercel.app
Source	github.com/AnimeshSatyapragyanPatra/blooddrop
Stack	React 19 · Express · Supabase Postgres + PostGIS · Vercel
Scale	11 tables · 43 API endpoints · 18 screens · 100 automated checks
Report generated	31 October 2026

1. The problem

When a hospital runs short of blood, the search today happens on WhatsApp. A message with a phone number is forwarded through groups until somebody responds. It fails in four specific ways: the forward keeps circulating long after the need is met; it reaches people by social proximity rather than by blood compatibility or distance; nobody can tell a real request from a prank; and the requester receives silence rather than a status, so they cannot tell whether anyone is coming.

BloodDrop replaces the forward with a system that matches on the things that actually matter and reports back.

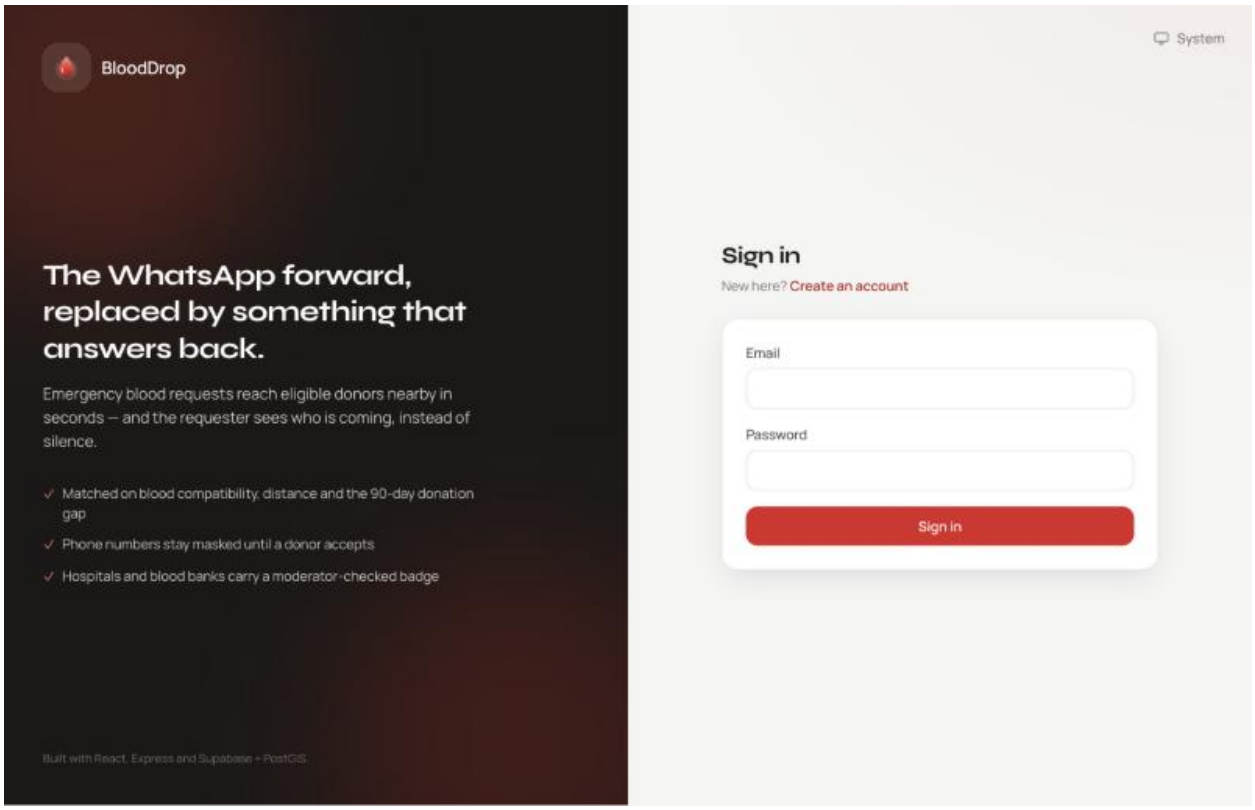


Figure 1 — Sign-in.

2. How it works

2.1 The core loop

- **A donor registers** with blood group, a location pinned on a map, and the date they last gave.
- **A requester posts** the group needed, units, hospital, urgency and an alert radius.
- **The system matches** — PostGIS finds donors inside the radius, the app filters to compatible blood groups, removes anyone inside the 90-day donation gap, and ranks by distance with a nudge for reliability.
- **Alerts dispatch** to the shortlist — capped so alerts stay meaningful.
- **Donors answer** "available" or "can't make it", so the requester sees a status instead of silence.
- **The donor books a slot** at a licensed blood bank, which is where blood is legally collected.
- **The blood bank records the outcome**, which resets the donor's 90-day clock automatically.

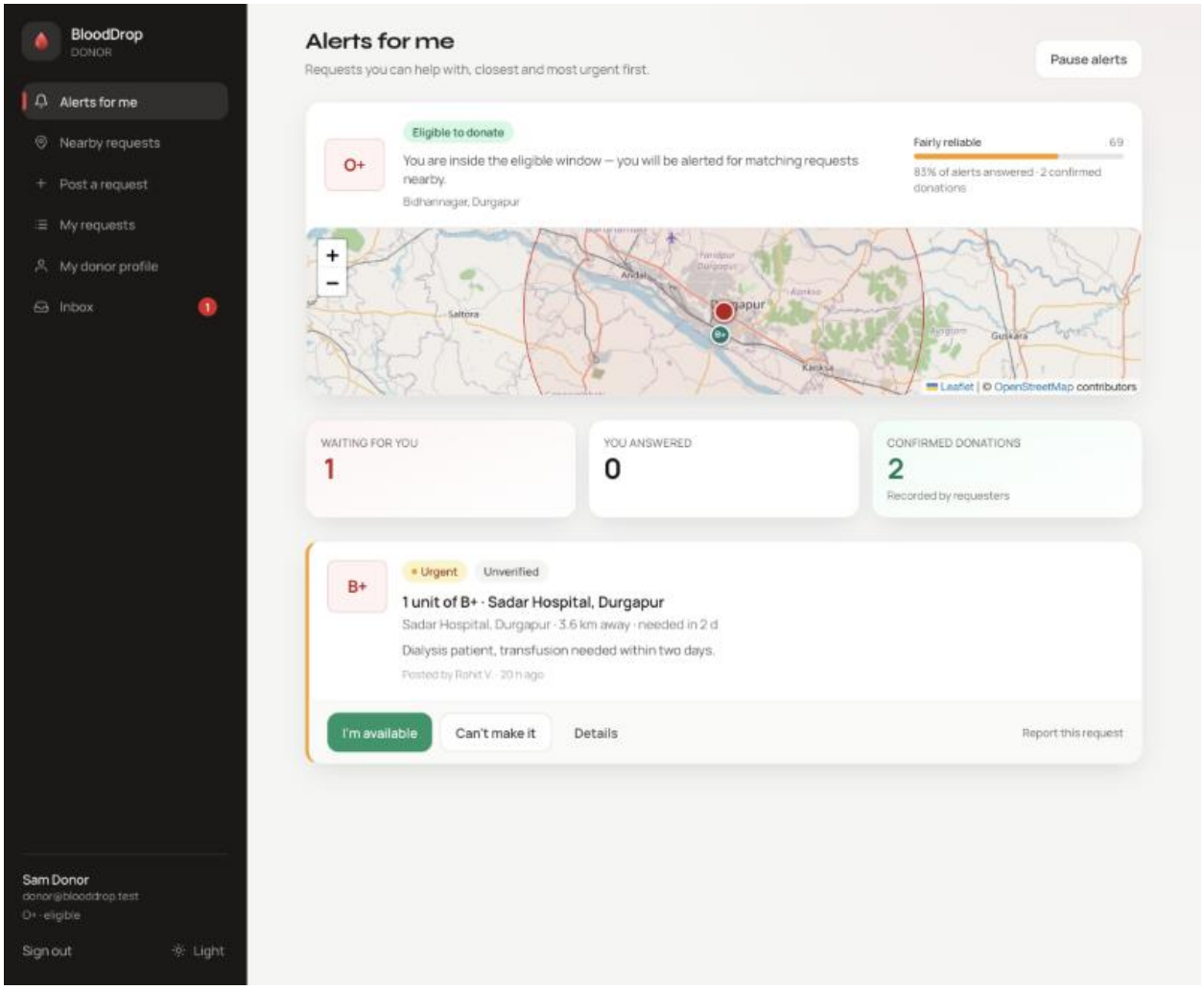


Figure 2 — A donor's alert feed: eligibility, reliability, and requests they can help with.

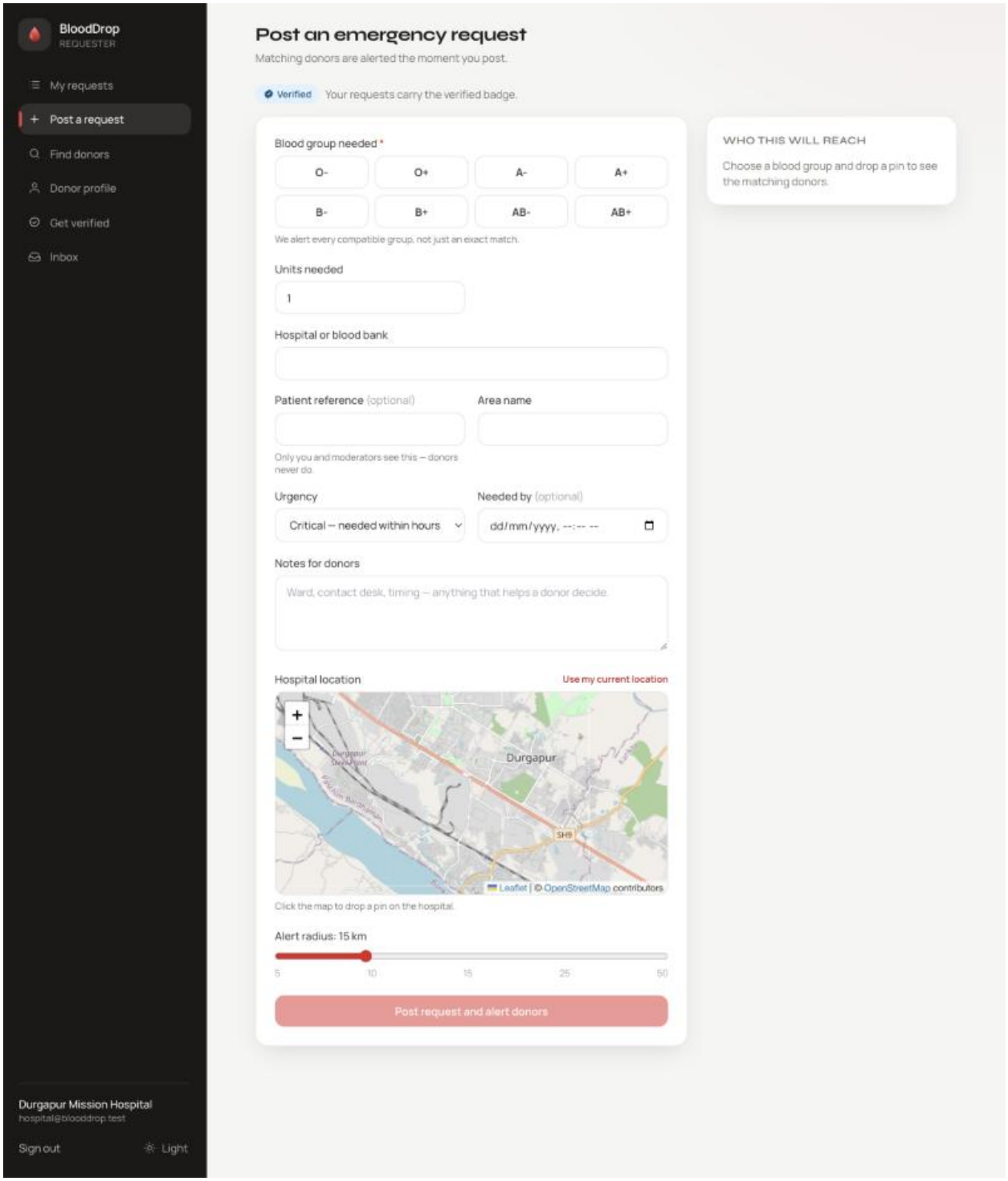


Figure 3 — Posting a request. The panel on the right counts matching donors live as the radius changes.

2.2 Matching rules

Three rules decide who is alerted, and all three are enforced on the server.

Rule	Why it matters
Red-cell compatibility, not equality	A request for A+ alerts O-, O+, A- and A+ donors. Matching on an exact string would leave the univers
The 90-day donation gap	A donor who gave recently is hidden from matching entirely, and cannot accept even if they try. Alertin
Distance first, reliability second	A donor 1 km away always outranks one 12 km away; between similar distances, the donor who actua

2.3 Choosing where to donate

Blood cannot pass from a donor to a patient directly — it must be collected at a licensed facility, screened and cross-matched first. The app therefore sends an accepting donor to a booked slot at a blood bank, and ranks the options for that donor: distance from them (not from the hospital), whether the bank is open, when it can next take them, and whether it is the bank that serves the patient. One option is marked *Suggested*, with the reasons shown rather than hidden in a score.

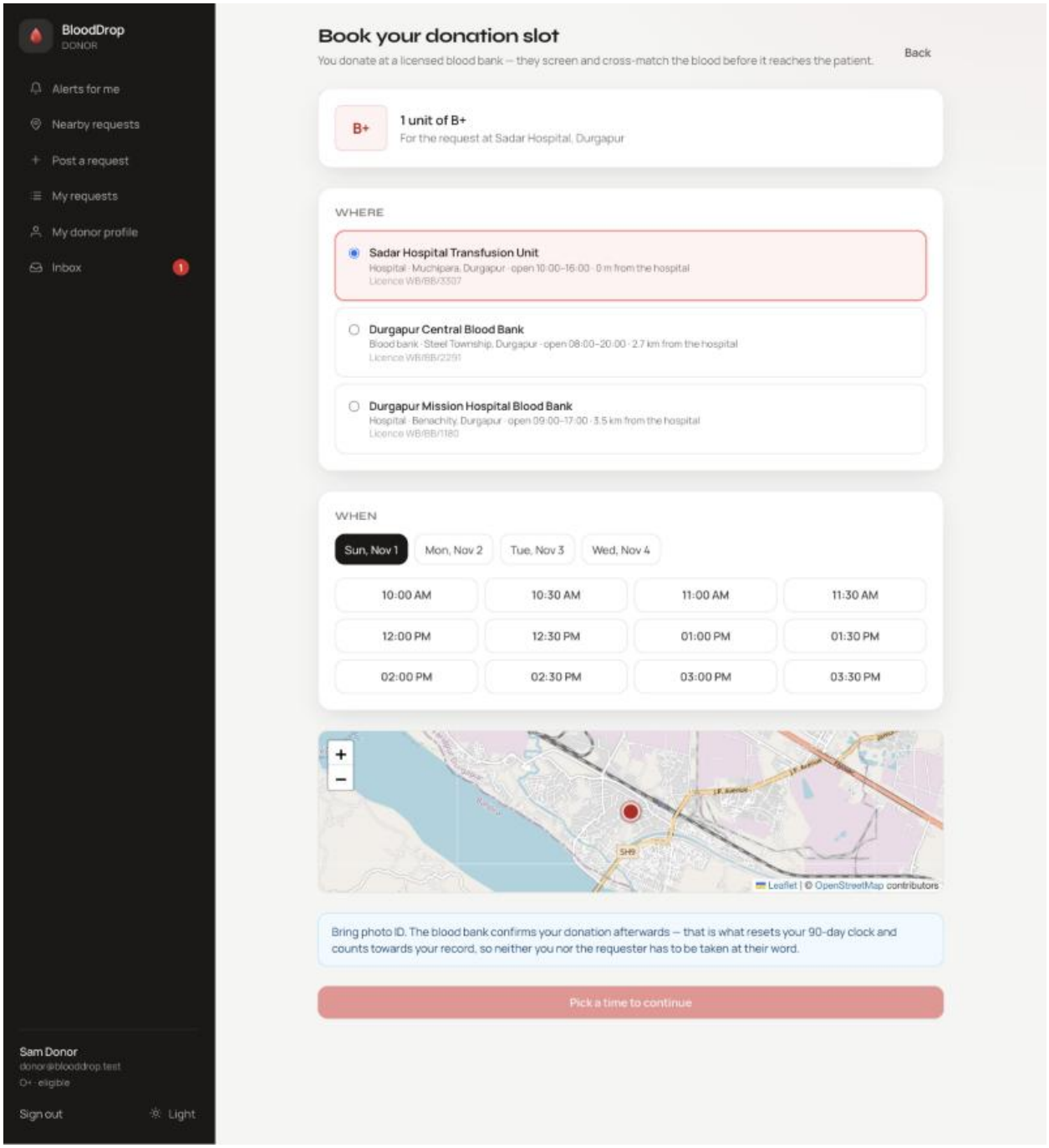


Figure 4 — Booking. Slots are derived from each facility's opening hours.

3. Trust and safety

An app that broadcasts medical need and personal contact details creates risks of its own. Five mechanisms address them.

Mechanism	What it prevents
Phone masking	Numbers show as +91 ••••• ••210 and unlock only between a donor who accepted and that requester. The
Verified requester badge	Hospitals submit a registration number which a moderator checks. It cannot be self-assigned, and it is a sn
Verified blood group	A donor can upload a donor card or lab report; a moderator confirms it. Requesters see what the group wa
Reliability score	Response rate and confirmed-donation rate, smoothed so a new donor starts neutral rather than at zero.
Report and moderation	Any request can be flagged. Removing one notifies the requester and every donor who responded, so nob

3.1 Who may confirm a donation

Earlier the requester marked a donor as having donated. That let them credit a friend or punish someone who did attend, which made the reliability score self-reported by an interested party. The blood bank is the only party that witnessed the donation, so confirmation now sits with them; a requester attempting it is refused and pointed at the bank.

3.2 Document retention

Uploaded proof is deleted as soon as a decision is recorded — the row keeps the outcome and the kind of evidence, not the scan. Holding identity documents past their purpose is a liability that grows with every upload and serves nothing once the review is done.

4. Architecture

4.1 Stack

Layer	Technology
Frontend	@supabase/supabase-js 2.112.3, leaflet 1.9.4, react 19.0.0, react-dom 19.0.0, react-leaflet 5.0.0, react-router-dom 7
Build	@vitejs/plugin-react, autoprefixer, postcss, tailwindcss, vite
Backend	@supabase/supabase-js 2.112.3, cors 2.8.5, dotenv 16.4.7, express 4.21.2
Database	Supabase Postgres with the PostGIS extension
Hosting	Vercel — static frontend plus one serverless function

4.2 Request path

Browser → Vercel static site → `/api/*` rewritten to a serverless function running the same Express app as local development → Supabase.

4.3 Domain logic

The rules live in small single-purpose modules rather than inside the route handlers, so each can be read and tested on its own.

Module	Responsibility
blood.js	red-cell compatibility — who can give to whom
demoData.js	the demo dataset, shared by mock and real seeding
eligibility.js	the 90-day donation gap and the eligible-from date
geo.js	great-circle distance
matching.js	compatibility + radius + eligibility + ranking
mockDb.js	an in-memory stand-in for Supabase so the app runs with no setup
notify.js	alert delivery across channels
privacy.js	name, phone and location masking
recommend.js	which blood bank this donor should go to, and why
reliability.js	smoothed response and show rates
slots.js	bookable times derived from opening hours
storage.js	document storage; local disk or Supabase Storage
supabase.js	client selection and the production safety guard
verification.js	what a requester is told about a verified blood group
views.js	row to API shape mappers

5. Database

11 tables, 13 indexes and 15 row-level-security policies, applied through 3 migrations.

Table	Holds
alerts	who was alerted for which request, and on what channel
appointments	a booked donation slot at a facility
donor_documents	proof of blood group and the review decision
donors	blood group, location, last donation, reliability counters
facilities	licensed blood banks: hours, capacity, licence number
notifications	the in-app inbox
profiles	every user: role, organisation, verified flag
reports	flags raised against a request or user
requests	an emergency request and its alert radius
responses	a donor's answer to an alert
verifications	hospital / blood bank badge applications

5.1 The spatial query

Distance is computed in the database, not in the application. The matching function uses `ST_DWithin` against a GiST index on a generated `geography` column, so the search stays fast as the donor table grows. `ST_Distance` alone would force a full scan.

```
and st_dwithin(d.geog,
  st_setsrid(st_makepoint(p_lng, p_lat), 4326)::geography,
  p_radius_km * 1000)
```

6. Running without any accounts

The repository contains a hand-written in-memory stand-in for Supabase — its auth, its query builder and the PostGIS matching function — so the whole application runs on a fresh clone with no account, no keys and no network. The controllers cannot tell the difference: they call `supabase.rpc("match_donors", …)` either way. It exists so the project can be demonstrated anywhere, and so the test suite needs no live database.

7. Testing

A single suite of 100 assertions boots the real Express application on an ephemeral port and exercises it over HTTP. It covers blood compatibility, radius filtering, eligibility, dispatch, the masking rules, booking, facility isolation, document review and moderation — including the negative cases: booking without accepting, booking a past or full slot, a donor reading another facility's queue, and a requester trying to confirm a donation the blood bank witnessed.

```
npm run smoke --prefix backend
```

8. Deployment

The repository root deploys to Vercel as-is. The frontend builds to a static site; `api/index.js` re-exports the same Express application as a serverless function. A deployment with missing Supabase credentials refuses to start rather than silently falling back to the in-memory database, which would otherwise look healthy while discarding every registration on each cold start.

9. Honest limitations

- Email and SMS are seams, not integrations — the in-app inbox is the channel that actually delivers.
- Facility opening hours are one window per day, with no holidays or per-day variation.
- Moderation is manual; there is no automated fraud detection.
- The engagement tier from the original plan — certificates, milestone badges, camp calendar, eligibility reminders — is not built, nor the predictive-shortage and WhatsApp-bridge ideas.
- Demo accounts exist for evaluation and would have to be removed before any real use.